

IF2211: Laporan Tugas Besar 2

Pemanfaatan Algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) pada Aplikasi Web DOM Tree



Dipersiapkan oleh:

13524110	Jennifer Khang
13524121	Billy Ontoseno Irawan
13524138	Ahmad Rlnofaros Muchtar

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025**

Daftar Isi

Deskripsi Tugas.....	4
1.1. Tags.....	4
1.2. CSS Selector.....	5
1.3. HTML Parsing.....	5
Landasan Teori.....	6
2.1. DOM Tree.....	6
2.2. CSS Selector.....	6
2.3. Traversal Graf.....	7
2.4. Breadth First Search.....	8
2.5. Depth First Search.....	9
2.6. Lowest Common Ancestor.....	10
Aplikasi BFS dan DFS.....	13
3.1. Struktur DOM Tree.....	13
3.2. Pemetaan Node.....	14
3.3. Implementasi BFS.....	14
3.4. Implementasi DFS.....	15
Implementasi dan Pengujian.....	16
4.1. Implementasi.....	16
4.2. Pengujian.....	22
4.3. Analisis Algoritma.....	31
Kesimpulan dan Saran.....	33
5.1. Kesimpulan.....	33
5.2. Saran.....	33
Lampiran.....	35
Tabel Penyelesaian.....	35
Daftar Pustaka.....	37

Daftar Gambar

Gambar 2.1 Contoh DOM Tree.....	7
Gambar 2.2. Contoh CSS Components.....	8
Gambar 2.3. Contoh Traversal Graf.....	9
Gambar 2.4. Contoh BFS.....	9
Gambar 2.5. Contoh DFS.....	10
Gambar 3.1. Struktur DOM Tree.....	11
Gambar 4.1.a. Source Code BFS.....	16
Gambar 4.1.b. Source Code DFS.....	16
Gambar 4.1.c. Source Code DLS.....	17
Gambar 4.1.d. Source Code HTML Parser.....	18
Gambar 4.1.e. Source Code CSS Selector.....	20
Gambar 4.2.a. Pengujian HTML File.....	20
Gambar 4.2.b. Pengujian URL.....	21
Gambar 4.2.c. Pengujian LCA.....	21
Gambar 4.2.d_1. Pengujian Tag Selector BFS.....	22
Gambar 4.2.d_2. Pengujian Class Selector BFS.....	22
Gambar 4.2.d_3. Pengujian IDSelector BFS.....	23
Gambar 4.2.d_4. Pengujian Universal Selector BFS.....	23
Gambar 4.2.d_5. Pengujian Child Combinator BFS.....	24
Gambar 4.2.d_6. Pengujian Descendant Combinator BFS.....	24
Gambar 4.2.d_7. Pengujian Adjacent Sibling Combinator BFS.....	25
Gambar 4.2.d_8. Pengujian General Sibling Combinator BFS.....	25
Gambar 4.2.e_1. Pengujian Tag Selector DFS.....	26
Gambar 4.2.e_2. Pengujian Class Selector DFS.....	26
Gambar 4.2.e_3. Pengujian ID Selector DFS.....	27
Gambar 4.2.e_4. Pengujian Universal Selector DFS.....	27
Gambar 4.2.e_5. Pengujian Child Combinator DFS.....	28
Gambar 4.2.e_6. Pengujian Descendant Combinator DFS.....	28
Gambar 4.2.e_7. Pengujian Adjacent Sibling Combinator DFS.....	29
Gambar 4.2.e_8. Pengujian General Sibling Combinator DFS.....	29

Bab 1

Deskripsi Tugas

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector. Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen yang penting dalam implementasi DOM Tree adalah sebagai berikut.

1.1. Tags

Tag HTML adalah penanda (markup) untuk menentukan struktur dan jenis elemen dalam halaman web. Ditulis dengan tanda kurung sudut (< >), biasanya memiliki tag pembuka dan penutup (misalnya <p></p>), meskipun ada juga yang *self-closing*. Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling.

1.2. CSS Selector

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector, memilih semua elemen berdasarkan tag.
- Class Selector (.class), memilih elemen dengan atribut class tertentu.
- ID Selector (#id), memilih elemen dengan id tertentu.
- Universal Selector (*), memilih semua elemen.
- Combinator, menggabungkan seluruh operasi selector yang terdiri dari child combinator, descendent combinator, adjacent combinator, dan general sibling combinator.

1.3. HTML Parsing

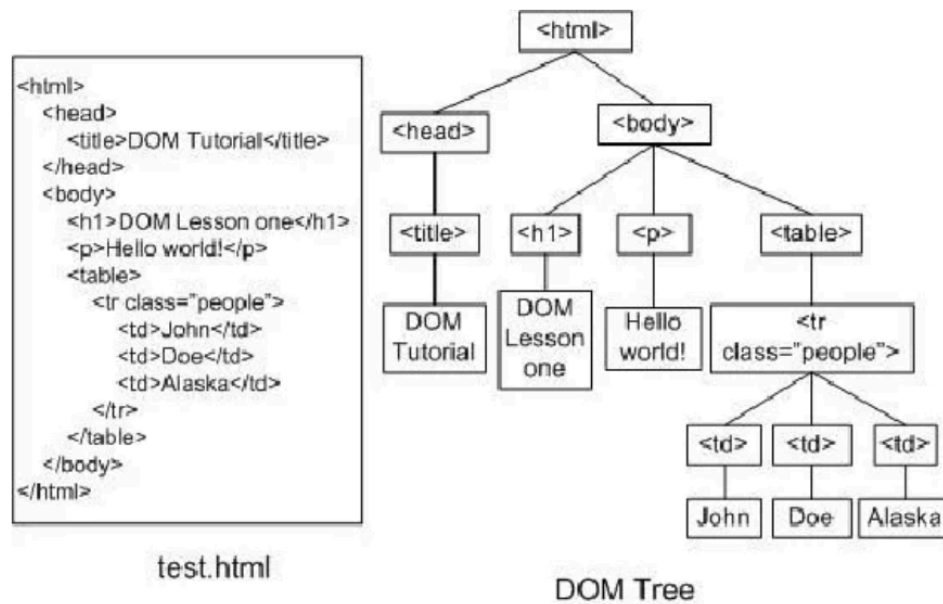
HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Setiap tag menjadi node dengan hubungan parent-child. Root untuk DOM Tree biasanya adalah <html>, yang bercabang ke <head> dan <body>. Struktur ini memungkinkan elemen ditelusuri dengan algoritma BFS atau DFS.

Bab 2

Landasan Teori

2.1. DOM Tree

Document Object Model adalah *application programming interface* (API) lintas *platform* yang menginterpretasi file HTML atau XML sebagai struktur *tree* dimana tiap simpul merupakan sebuah objek yang merepresentasikan suatu bagian dari dokumen file tersebut. Metode yang terdapat pada DOM memberikan akses untuk mengubah struktur, presentasi, dan kandungan dokumen.



Gambar 2.1 Contoh DOM Tree

2.2. CSS Selector

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

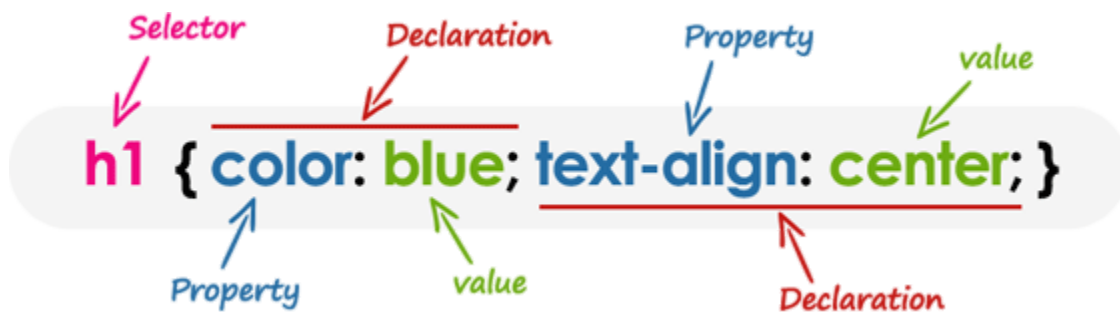
- Tag Selector: Memilih elemen berdasarkan nama tag HTML. Contoh: `p` memilih semua elemen `<p>`.
- Class Selector: Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (`.`). Contoh: `.box`

- ID Selector: Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#). Contoh: #header

- Universal Selector: Memilih semua elemen HTML. Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (" ")
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)



Gambar 2.2. Contoh CSS Components

2.3. Traversal Graf

Algoritma traversal graf adalah metode penelusuran simpul dalam graf dengan cara sistematis berdasarkan asumsi bahwa graf terhubung. Terdapat dua jenis pencarian solusi berbasis graf.

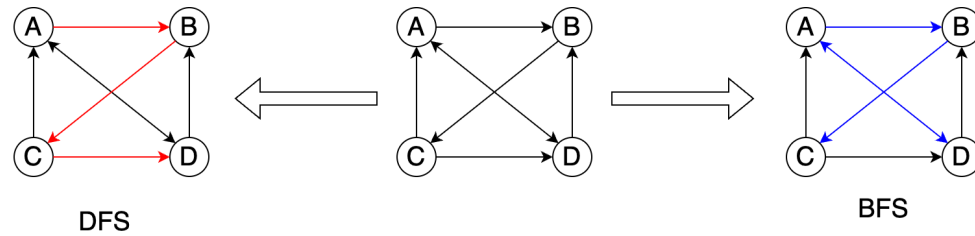
1. Dengan informasi (informed Search)

- Pencarian berbasis heuristik
- Menggunakan informasi tambahan berupa heuristic function untuk memperkirakan seberapa dekat suatu simpul ke tujuan, atau dengan kata lain simpul mana yang “lebih menjanjikan” ke simpul tujuan.
- Contoh algoritma: Greedy Best First Search, A*

2. Tanpa informasi (uninformed/blind search)

- Tidak ada informasi tambahan yang disediakan tentang seberapa dekat suatu simpul dengan tujuan (goal)
- Satu-satunya informasi hanya dari graf itu sendiri (graf berbobot)

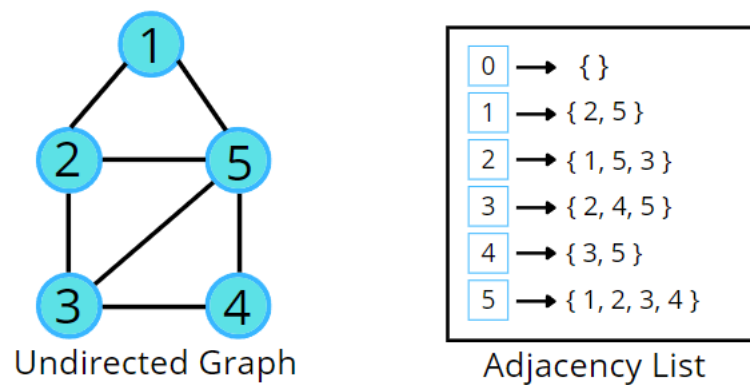
- Contoh algoritma: DFS, BFS, Depth Limited Search, Iterative Deepening Search, Uniform Cost Search



Gambar 2.3. Contoh Traversal Graf

2.4. Breadth First Search

Breadth First Search (BFS) adalah algoritma pencarian simpul yang memenuhi syarat pada sebuah graf terhubung berdasarkan jarak dari simpul akarnya[1]. Diberikan suatu graf $G = (V, E)$ dan simpul akar r , algoritma breadth-first search secara sistematis menelusuri ujung dari G untuk mencari setiap simpul yang terjangkau dari r [2].



Gambar 2.4. Contoh BFS

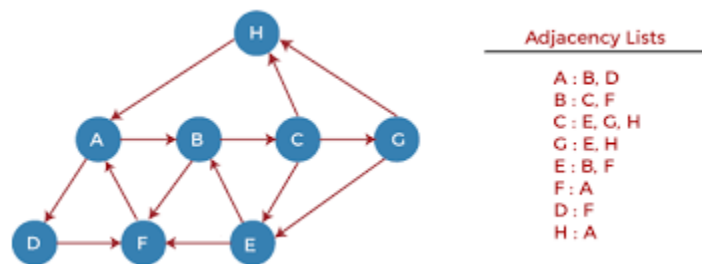
Berikut uraian tahapan BFS secara algoritmik.

1. Inisialisasi: Buat antrian (queue) yang berisi simpul akar (root) sebagai titik awal.
2. Eksplorasi: Selama queue tidak kosong:
 - a. Dequeue simpul pertama sebagai simpul saat ini dan kunjungi
 - b. Masukkan tetangga dalam queue untuk tiap tetangga yang belum dikunjungi simpul saat ini

- c. Tandai simpul saat ini sebagai telah dikunjungi
3. Basis: Ulangi eksplorasi hingga queue kosong atau simpul tujuan telah dikunjungi

2.5. Depth First Search

Depth First Search (DFS) adalah algoritma pencarian simpul pada sebuah graf yang menelusuri suatu cabang dalam graf tersebut sejauh mungkin sampai simpul daun (ujung) sebelum *backtracking* ke simpul terdekat yang bercabang ke simpul yang belum ditelusuri.



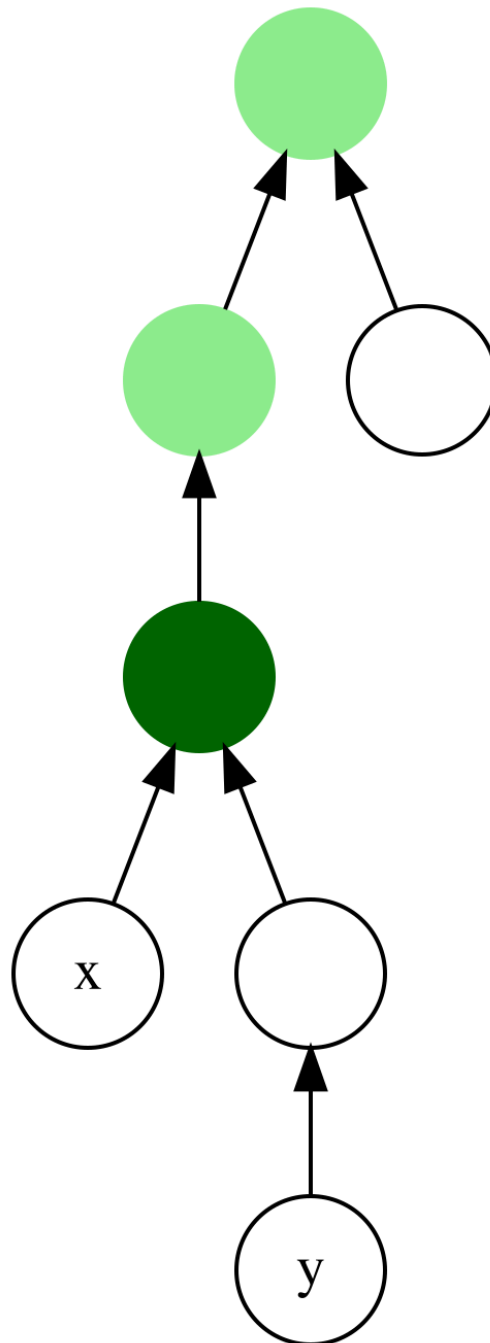
Gambar 2.5. Contoh DFS

Berikut uraian tahapan DFS secara algoritmik:

1. Inisialisasi: Buat stack yang berisi simpul akar (root) sebagai titik awal.
2. Eksplorasi: Selama stack belum kosong:
 - a. Pop simpul dari stack sebagai simpul saat ini dan kunjungi
 - b. Masukkan simpul pertama yang bertetangga dengan simpul saat ini ke dalam stack
 - c. Tandai simpul saat ini sebagai telah dikunjungi
3. Backtrack: Ketika mencapai simpul tanpa tetangga yang tidak telah dikunjungi, kunjungi simpul terakhir yang telah dikunjungi dan masih memiliki tetangga yang tidak telah dikunjungi.
4. Basis: Ulangi eksplorasi dan backtracking sampai stack kosong (simpul habis) atau simpul tujuan telah dikunjungi

2.6. Lowest Common Ancestor

Lowest Common Ancestor (LCA) dari dua simpul v dan w dalam sebuah graf T merupakan simpul terjauh dari simpul akar yang memiliki v dan w sebagai keturunannya. LCA juga mendefinisikan sebuah simpul sebagai keturunannya sendiri sehingga keturunan langsung w dapat menentukan LCA sebagai simpul v .



Gambar 2.6. Contoh LCA

Berikut uraian tahapan LCA secara algoritmik

1. Tahap 1: Preprocessing
 - a. Inisialisasi Tabel Binary Lifting
 1. Hitung ukuran *tree* untuk menentukan nilai maksimum pangkat yang diperlukan ($\log BL$)
 2. Buat struktur data Tabel Ancestor(up), kedalaman simpul(depth), dan pemetaan ID ke simpul (nodeByUniqueID)
 - b. DFS untuk mengisi Parent Langsung
 1. Lakukan DFS dari akar
 2. Untuk setiap simpul:
 - a. Simpan uniqueID, catat kedalaman simpul dan ancestor ke-0 di tabel ancestor
 3. Rekursif untuk semua child dengan kedalaman +1
 - c. Mengisi Tabel Binary Lifting
 1. Untuk setiap simpul dalam *tree*:
 - a. Dapatkan midAncestor (Ancestor $2^{(k-1)}$ dari simpul)
 - b. Jika midAncestor ada, dapatkan midAncestor darinya
 - c. Simpan sebagai ancestor ke 2^k dari simpul saat ini
2. Tahap 2: Query LCA
 - a. Validasi Input
 1. Periksa kedua simpul agar tidak null
 2. Ambil uniqueID kedua simpul
 3. Jika ID sama, simpul tersebut adalah LCA
 - b. Menyamakan kedalaman
 1. Bandingkan kedalaman kedua simpul dengan depth
 2. Tentukan simpul yang lebih dalam dan dangkal
 3. Hitung selisih kedalaman
 4. Naikkan simpul yang lebih dalam sebanyak selisih
 - c. Mencari LCA

1. Jika setelah penyamaan kedalaman kedua simpul sama, kembalikan simpul sebagai LCA
2. Jika tidak, dari pangkat tinggi ke rendah, lakukan:
 - a. Dapatkan ancestor ke 2^k dari kedua simpul
 - b. Jika ancestor berbeda naikkan kedua simpul ke ancestor masing-masing
3. Setelah loop selesai, LCA adalah parent langsung dari kedua simpul

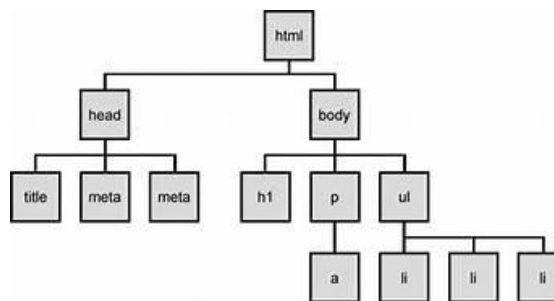
Bab 3

Aplikasi BFS dan DFS

Bab ini akan menjelaskan kerangka berpikir implementasi melalui pseudocode dari *source code*.

3.1. Struktur DOM Tree

DOM Tree sendiri merupakan representasi struktur dari akar-akar dokumen HTML atau XML. Setiap HTML direpresentasikan sebagai sebuah node, dengan hubungan hierarkis berupa parent, child, dan sibling. Node paling atas disebut sebagai root, umumnya adalah <HTML>. Dalam implementasi ini, setiap node pada DOM Tree memiliki informasi elemen, daftar child, serta relasi dengan node lain. Selain itu, setiap node juga menyimpan informasi tambahan seperti kedalaman (depth) dan elemen visualisasi untuk mendukung proses rendering. Struktur Tree menyimpan root serta informasi tambahan seperti kedalaman maksimum dari pohon.



Gambar 3.1. Struktur DOM Tree

Berikut ini merupakan implementasi untuk struktur Node beserta Tree-nya.

```
export class Tree{
  root: Node | null;
  maxDepth: number = 0;}

export class Node {
  tag: string;
  attributes: Record<string, string>;
  parent: Node | null;
  children: Node[] = [];
  depth: number = 0;
```

```
el?: SVGCircleElement;  
edgeEl?: SVGLineElement;}
```

3.2. Pemetaan Node

Pemetaan node dilakukan dengan mengubah dokumen HTML hasil scraping menjadi struktur DOM Tree. Setiap tag HTML akan diparsing dan direpresentasikan sebagai node, kemudian disusun berdasarkan hubungan parent-child sesuai struktur HTML. Proses ini memastikan bahwa struktur asli dokumen tetap terjaga, sehingga traversal dapat dilakukan dengan benar. Implementasinya menggunakan library `htmlparser2` yang *tokenize* tag secara langsung. Selain itu, setiap node juga disesuaikan lagi atribut tambahannya seperti *depth* ketika *parsing* untuk menunjukkan tingkat kedalaman dalam tree.

3.3. Implementasi BFS

Algoritma Breadth First Search adalah algoritma pencarian yang melebar. Pencarian dimulai dari sebuah simpul dan menelusuri simpul *adjacent*-nya. Setelah semua simpul yang berdekatan dikunjungi, maka simpul-simpul yang berdekatan tersebut akan ditelusuri. BFS menelusuri semua simpul pada kedalaman tertentu terlebih dahulu sebelum menelusuri simpul pada kedalaman selanjutnya. Tim mengaplikasikan BFS dengan memanfaatkan struktur queue, di mana tiap anak dari suatu *parent* akan ditelusuri satu per satu. Berikut adalah tahapan lengkapnya.

1. Algoritma memasukkan simpul sumber *v* yang diberikan ke dalam queue dan tandai sebagai telah dikunjungi.
2. Saat queue tidak kosong, suatu node akan diambil dan dikunjungi node tersebut. Node *adjacent* akan dimasukkan ke dalam queue untuk setiap tetangga yang belum dikunjungi dari simpul yang dikeluarkan. Simpul yang sudah diambil akan dianggap sebagai simpul yang sudah dikunjungi.
3. Tahapan 1 dan 2 diulangi kembali hingga seluruh node selesai diimplementasi.

3.4. Implementasi DFS

Algoritma Depth First search merupakan algoritma pencarian yang bekerja dengan menelusuri sebuah node secara mendalam. Dalam tugas ini, DFS diimplementasikan menggunakan pendekatan rekursif atau stack. Setiap node yang dikunjungi akan langsung diperiksa kesesuaiannya terhadap CSS selector. Berikut merupakan tahapan lengkapnya.

1. Algoritma memasukan simpul yang diberikan ke dalam stack dan ditandai sebagai node yang sudah dikunjungi.
2. Ketika stack tidak kosong, satu simpul dikeluarkan dari stack dan mengunjungi node tersebut. Node akan backtrack dan mengunjungi node lainnya dan dimasukkan ke dalam stack. Tahapan diulangi hingga mencapai noe target.

Perhatikan bahwa algoritma ini memanfaatkan pencarian runut-balik untuk kembali ke simpul pada kedalaman sebelumnya. Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi atau ditemukan simpul target.

3.5. Implementasi LCA

Lowest Common Ancestor (LCA) merupakan algoritma yang digunakan untuk menentukan simpul leluhur terendah yang sama dari dua node dalam suatu pohon. LCA pada kasus ini digunakan untuk menentukan elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu. Berikut merupakan implementasinya.

Bab 4

Implementasi dan Pengujian

4.1. Implementasi

a. Arsitektur Sistem

Arsitektur sistem terdiri dari dua bagian utama: backend dan frontend. Backend bertanggung jawab untuk memproses input HTML atau URL (dan melakukan scraping menggunakan Axios), menghasilkan struktur pohon DOM, dan mengembalikan data JSON yang disesuaikan untuk D3.js yang merepresentasikan pohon tersebut. Frontend bertugas untuk mengirimkan input, menerima data dari backend, memvisualisasikan pohon DOM menggunakan komponen TreeView, dan menyediakan antarmuka pengguna untuk menjalankan traversal DOM dengan berbagai mode dan konfigurasi. Interaksi serta struktur folder antara Frontend dan Backend menggunakan sistem Data Transfer Object.

i. Frontend

Sistem frontend dikembangkan menggunakan TypeScript dengan framework bun dengan vite serta library React. Meskipun penggunaan Next.js dapat dianggap cukup berlebih untuk proyek ini, pemilihan tersebut tetap dilakukan sesuai dengan spesifikasi tugas. Selain itu, karena aplikasi hanya memiliki satu halaman utama, fitur-fitur lanjutan seperti App Routing tidak dimanfaatkan secara maksimal.

ii. Backend

Sistem backend juga dikembangkan menggunakan TypeScript, sehingga seluruh aplikasi memiliki keseragaman dalam bahasa pemrograman. Backend bertanggung jawab untuk menangani logika utama sistem, seperti web scraping, parsing HTML menjadi DOM Tree, serta implementasi algoritma traversal seperti BFS, DFS, dan DLS. Backend menggunakan node.js

iii. Dockerization

Dockerization merupakan proses pengemasan aplikasi ke dalam container agar dapat dijalankan secara konsisten di berbagai lingkungan, seperti tahapan development, staging, maupun production. Docker membungkus seluruh dependensi, konfigurasi, dan runtime dalam satu image, sehingga setiap *environment* dapat di-setup semestinya. Untuk mempermudah proses build dan menjalankan keseluruhan sistem, digunakan docker-compose sebagai pembangun setup. Docker Compose memungkinkan pendefinisian dan pengelolaan beberapa container sekaligus melalui satu file konfigurasi. Docker file yang diimplementasi menggunakan multi-stage build dengan image dasar oven/bun:1, di mana tahap berikutnya dibagi menjadi dua, yakni backend dan frontend. Backend dijalankan pada folder /src/backend dengan expose port 3000 sementara Frontend dijalankan di src/frontend dengan expose port 5173.

b. Alur Sistem secara Ringkas

Pengguna memilih mode input (HTML atau URL) dan memberikan input yang sesuai. Ketika tombol "Parse" ditekan, frontend mengirimkan permintaan ke endpoint backend yang relevan (/api/parse untuk HTML atau /api/url untuk URL). Backend memproses input, membangun pohon DOM, dan mengembalikan data JSON yang mencakup struktur pohon dan kedalaman maksimum. Data ini kemudian diubah menjadi objek Node di frontend dan divisualisasikan menggunakan TreeView. Pengguna dapat memasukkan query CSS selector dan memilih algoritma traversal untuk menjalankan pencarian. Selama traversal, setiap node yang dikunjungi dan cocok dengan query akan disorot, dan log traversal ditampilkan di bawah graf Tree yang telah dibangkitkan. Waktu eksekusi dan jumlah node yang dikunjungi juga dihitung dan ditampilkan.

c. Source Code

i. BFS



```

1  async bfsAnimated(root: Node, query: string, delay: number, onVisit?: (node: Node) => void, onMatch?: (node: Node) => void) {
2      const queue: Node[] = [root];
3
4      while (queue.length > 0) {
5          const current = queue.shift();
6
7          onVisit?.(current);
8
9          if (this.selector.matchQuery(current, query)) {
10             onMatch?.(current);
11         }
12
13         await new Promise(r => setTimeout(r, delay));
14
15         for (const child of current.children) {
16             queue.push(child);
17         }
18     }
19 }

```

Gambar 4.1.a. Source Code BFS

ii. DFS



```

1  async dfsAnimated(node: Node, query: string, delay: number, onVisit?:
2      (node: Node) => void, onMatch?: (node: Node) => void, result: Node[] = []): Promise<Node[]> {
3      onVisit?.(node);
4
5      if (this.selector.matchQuery(node, query)) {
6          result.push(node);
7          onMatch?.(node);
8      }
9
10     await new Promise(res => setTimeout(res, delay));
11
12     for (const child of node.children) {
13         await this.dfsAnimated(child, query, delay, onVisit, onMatch, result);
14     }
15
16     return result;
17 }
18

```

Gambar 4.1.b. Source Code DFS

iii. DLS

```

1  async dlsAnimated(node: Node, query: string, maxDepth: number, minDepth: number,
2      currentDepth: number, delay: number, onVisit?: (node: Node) => void, onMatch?:
3      (node: Node) => void) {
4
5      onVisit?.(node);
6
7      if (
8          currentDepth >= minDepth &&
9          this.selector.matchQuery(node, query)
10     ) {
11         onMatch?.(node);
12     }
13
14     await new Promise(r => setTimeout(r, delay));
15
16     if (currentDepth >= maxDepth) return;
17
18     for (const child of node.children) {
19         await this.dlsAnimated(
20             child,
21             query,
22             maxDepth,
23             minDepth,
24             currentDepth + 1,
25             delay,
26             onVisit,
27             onMatch
28         );
29     }
30 }
31 }

```

Gambar 4.1.c. Source Code DLS

iv. HTML Parser

```

1  export class HTMLParser {
2      parse(html: string): Tree {
3          const stack: Node[] = [];
4          let root: Node | null = null;
5
6          const parser = new Parser(
7              {
8                  // Attribute returns in Record<string, string>,
9                  // bisa pakai metode hasClass untuk pencarian tag class
10                 onopentag: (name: string, attributes: any) => {
11                     const newNode = new Node(name, attributes);
12
13                     if (stack.length === 0) {
14                         root = newNode;
15                     } else {
16                         const parent = stack[stack.length - 1];
17                         if (parent) {
18                             parent.addChild(newNode);
19                         }
20                     }
21                     if (!SELF_CLOSING.has(name)) {
22                         stack.push(newNode);
23                     }
24                 },
25
26                 onclosetag: (name: string) => {
27                     if (SELF_CLOSING.has(name)) {
28                         stack.pop();
29                     }
30                 },
31             },
32             {
33                 decodeEntities: true
34             }
35         );
36
37         parser.write(html);
38         parser.end();
39
40         if (!root) {
41             throw new ParserError("[Parser] Gagal memuat DOM Tree");
42         }
43
44         return new Tree(root);
45     }
46 }

```

Gambar 4.1.d. Source Code HTML Parser

v. CSS Selector

```

1
2 matchQuery(node: Node, selector: string): boolean {
3   if (!selector) return false;
4
5   // Child combinator
6   const childSplit = this.splitLast(selector, ">");
7   if (childSplit) {
8     const [parentPart, childPart] = childSplit;
9     return node.parent !== null && this.matchQuery(node.parent, parentPart) && this.matchBasic(node, childPart);
10  }
11
12  const adjacentSplit = this.splitLast(selector, "+");
13  if (adjacentSplit) {
14    const [siblingPart, nodePart] = adjacentSplit;
15    return node.previousSibling !== null && this.matchQuery(node.previousSibling, siblingPart) && this.matchBasic(node, nodePart);
16  }
17
18  const siblingSplit = this.splitLast(selector, "~");
19  if (siblingSplit) {
20    const [siblingPart, nodePart] = siblingSplit;
21    return node.siblings.some(sib => this.matchQuery(sib, siblingPart)) && this.matchBasic(node, nodePart);
22  }
23
24  const descendantSplit = this.splitLast(selector, " ");
25  if (descendantSplit) {
26    const [ancestorPart, nodePart] = descendantSplit;
27    return node.hasAncestor(n => this.matchQuery(n, ancestorPart)) && this.matchBasic(node, nodePart);
28  }
29
30  return this.matchBasic(node, selector);
31 }
32

```

```

1 matchBasic(node: Node, selector: string): boolean {
2   if (selector === "*" ) return true;
3
4   if (selector.startsWith("#")) {
5     return node.id === selector.slice(1);
6   }
7
8   // Multi-class
9   if (selector.startsWith(".")) {
10    return selector.slice(1).split(".").every(c => node.hasClass(c));
11  }
12
13  // Attribute selector
14  if (selector.includes("[")) {
15    const attrMatch = selector.match(/^(\w+)?\[([\w+](?:=([.+]?)\w+)?$)/);
16    if (!attrMatch) return false;
17    const [, tag, attr, value] = attrMatch;
18    if (!attr) return false;
19    if (tag && node.tag !== tag) return false;
20    return node.getAttribute(attr) === value;
21  }
22
23  // tag.class
24  const tagClassMatch = selector.match(/^(\w+)\.([\w+])$/);
25  if (tagClassMatch) {
26    const [, tag, cls] = tagClassMatch;
27    if (!cls) return false;
28    return node.tag === tag && cls.split(".").every(c => node.hasClass(c));
29  }
30
31  return node.tag === selector;
32 }
33

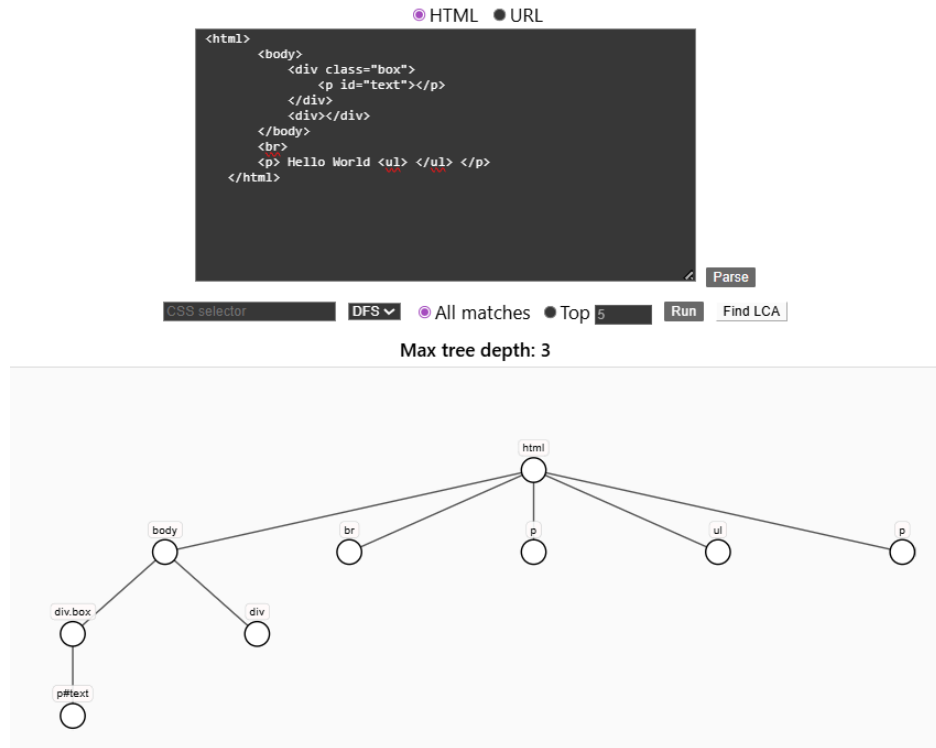
```

Gambar 4.1.e. Source Code CSS Selector

4.2. Pengujian

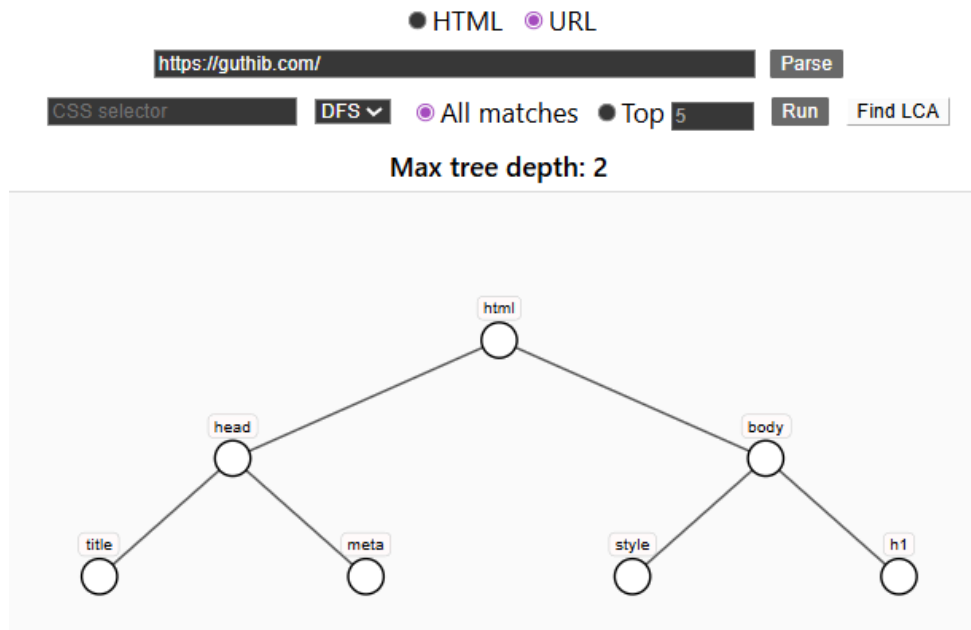
Berikut merupakan beberapa pengujian dari implementasi source code.

a. Pengujian dengan HTML File



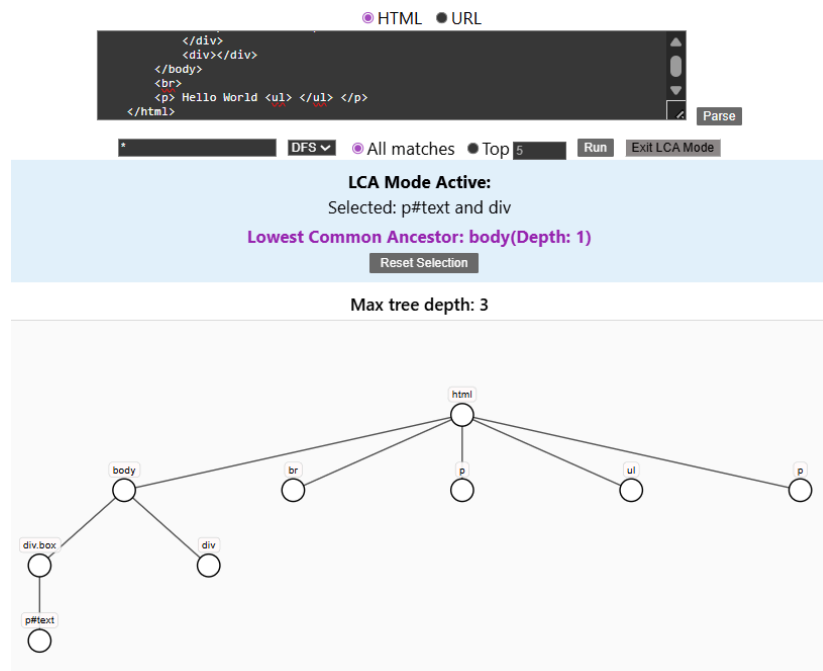
Gambar 4.2.a. Pengujian HTML File

b. Pengujian dengan URL



Gambar 4.2.b. Pengujian URL

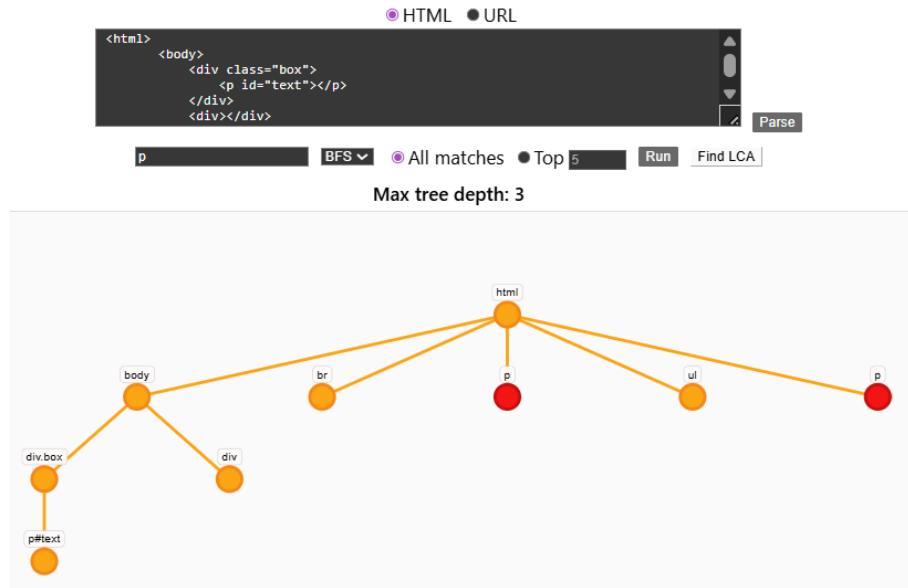
c. LCA



Gambar 4.2.c. Pengujian LCA

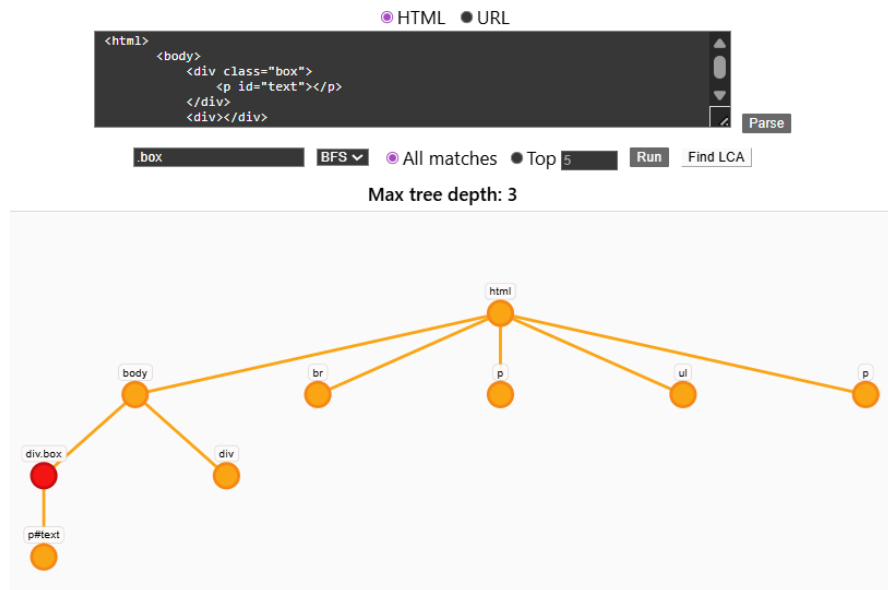
d. Pencarian Tag dengan BFS

i. Tag Selector



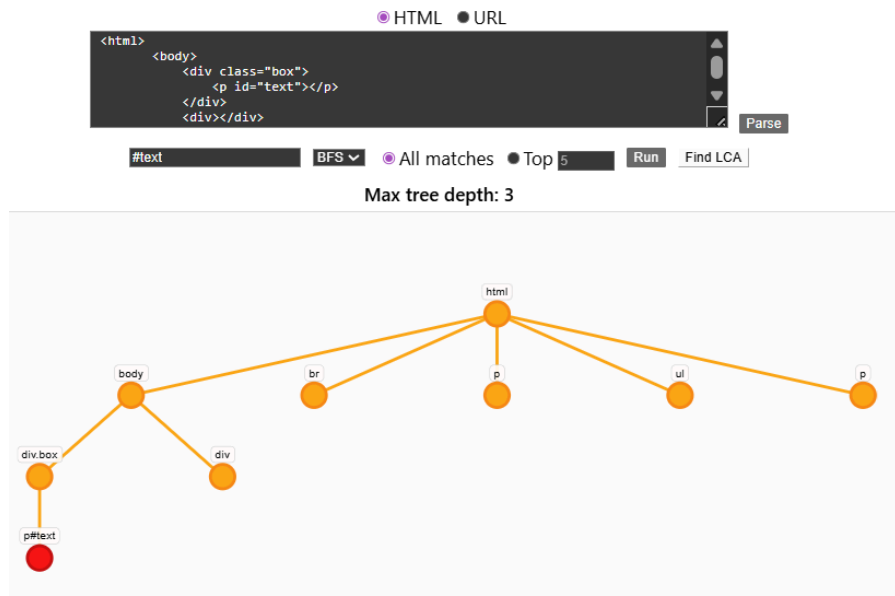
Gambar 4.2.d_1. Pengujian Tag Selector BFS

ii. Class Selector



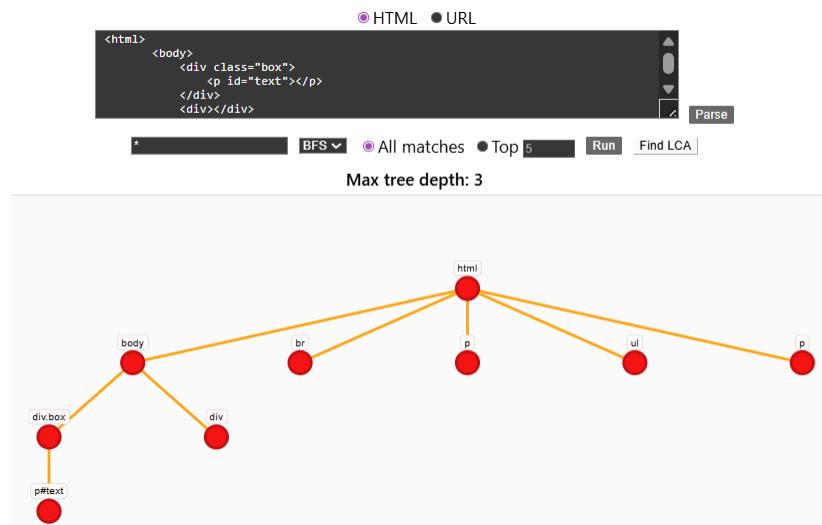
Gambar 4.2.d_2. Pengujian Class Selector BFS

iii. ID Selector



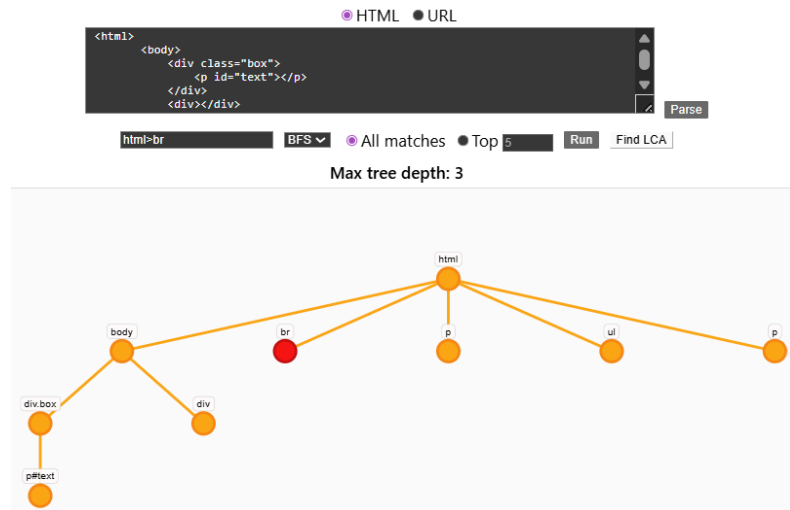
Gambar 4.2.d_3. Pengujian IDSelector BFS

iv. Universal Selector



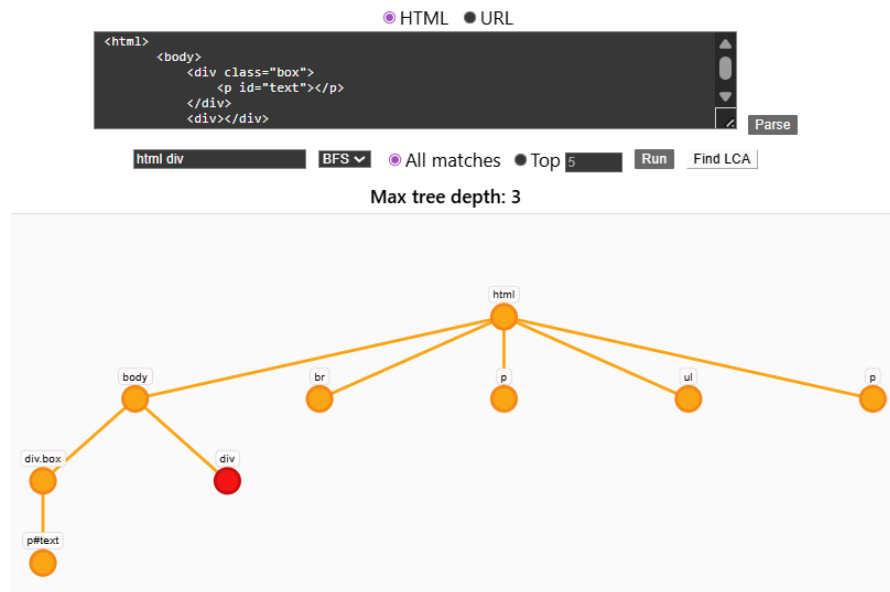
Gambar 4.2.d_4. Pengujian Universal Selector BFS

v. Child Combinator



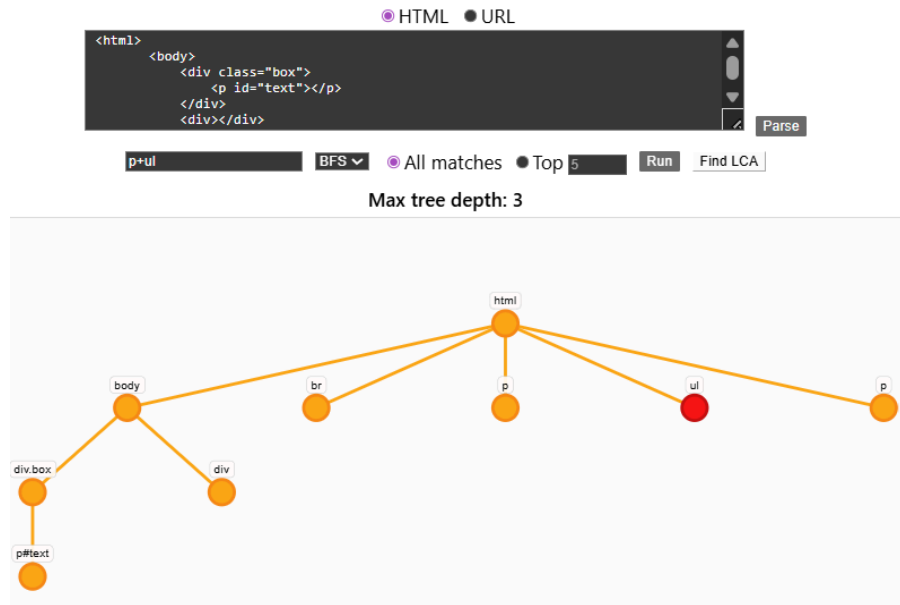
Gambar 4.2.d_5. Pengujian Child Combinator BFS

vi. Descendant Combinator



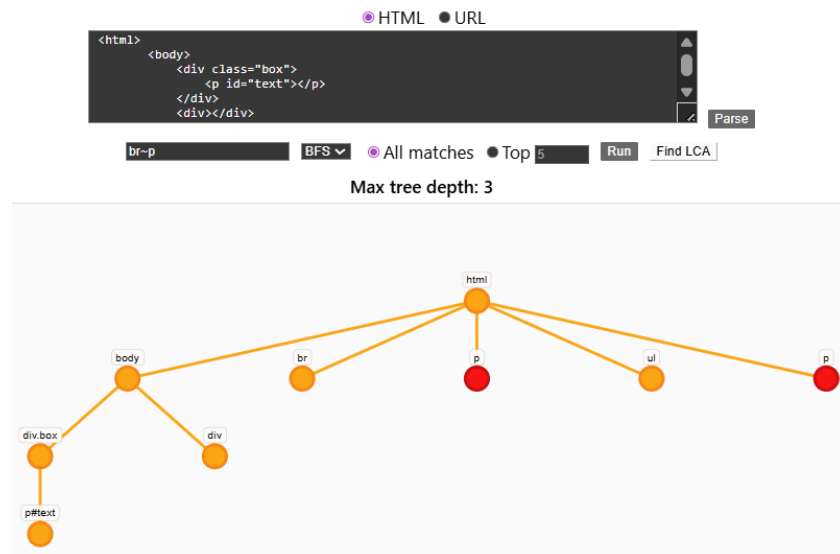
Gambar 4.2.d_6. Pengujian Descendant Combinator BFS

vii. Adjacent Sibling Combinator



Gambar 4.2.d_7. Pengujian Adjacent Sibling Combinator BFS

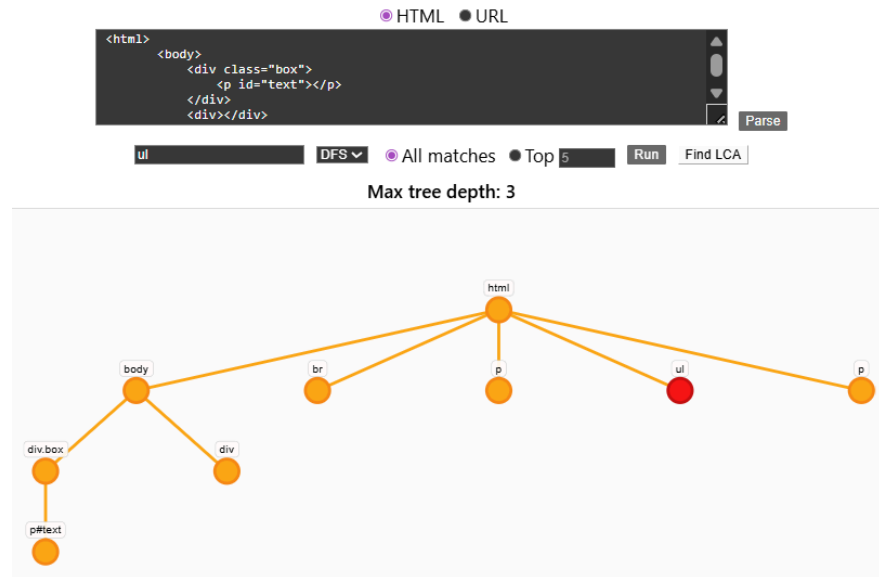
viii. General Sibling Combinator



Gambar 4.2.d_8. Pengujian General Sibling Combinator BFS

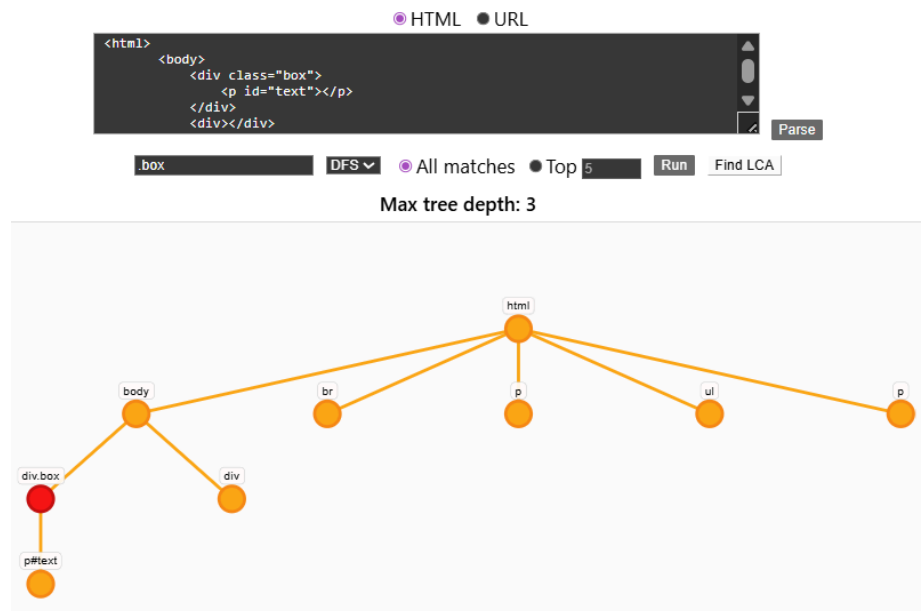
e. Pencarian Tag dengan DFS

i. Tag Selector



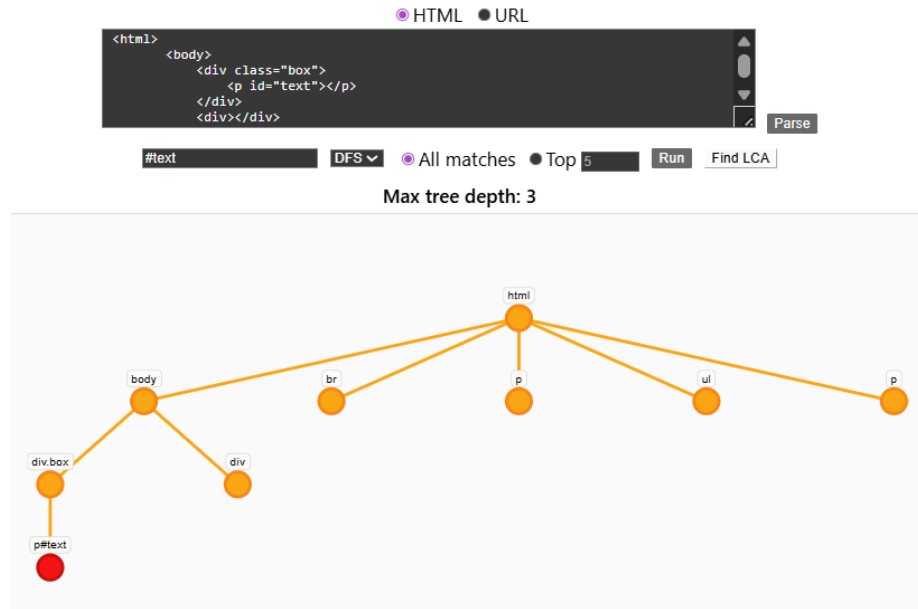
Gambar 4.2.e_1. Pengujian Tag Selector DFS

ii. Class Selector



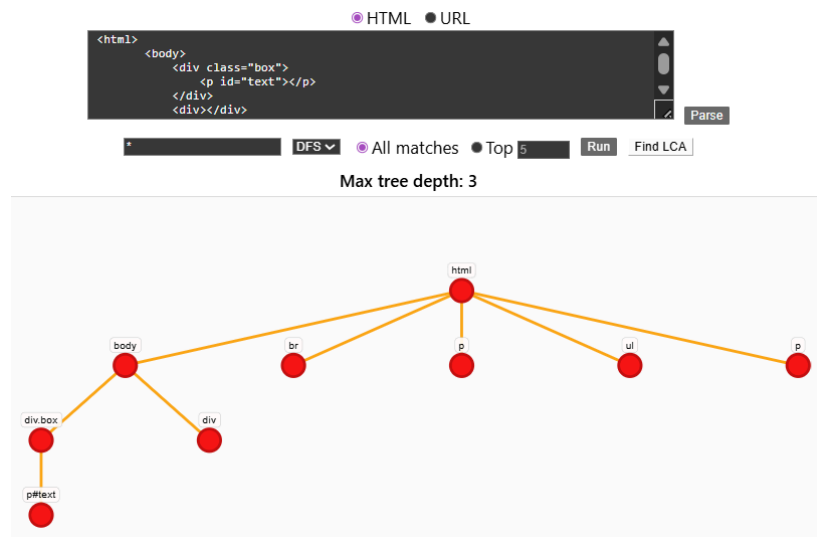
Gambar 4.2.e_2. Pengujian Class Selector DFS

iii. ID Selector



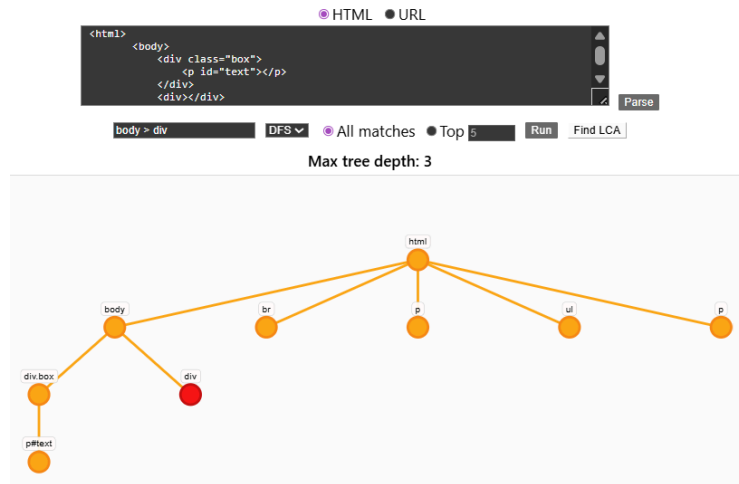
Gambar 4.2.e_3. Pengujian ID Selector DFS

iv. Universal Selector



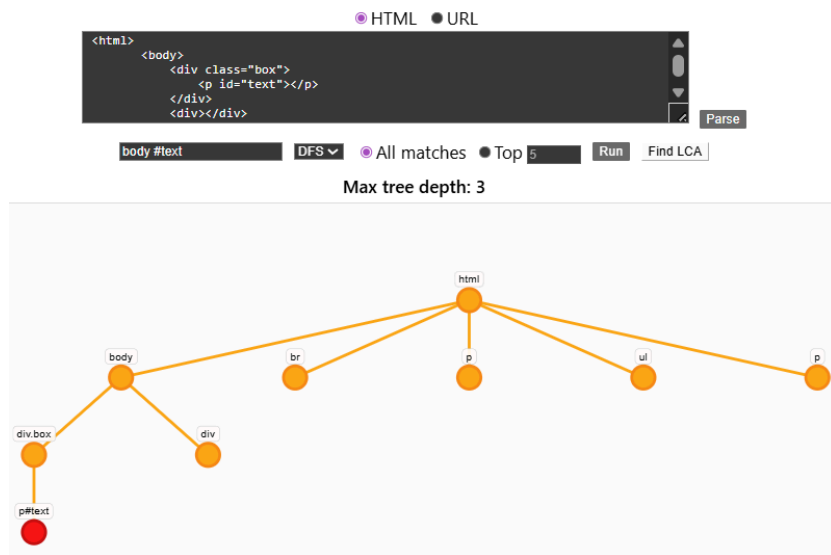
Gambar 4.2.e_4. Pengujian Universal Selector DFS

v. Child Combinator



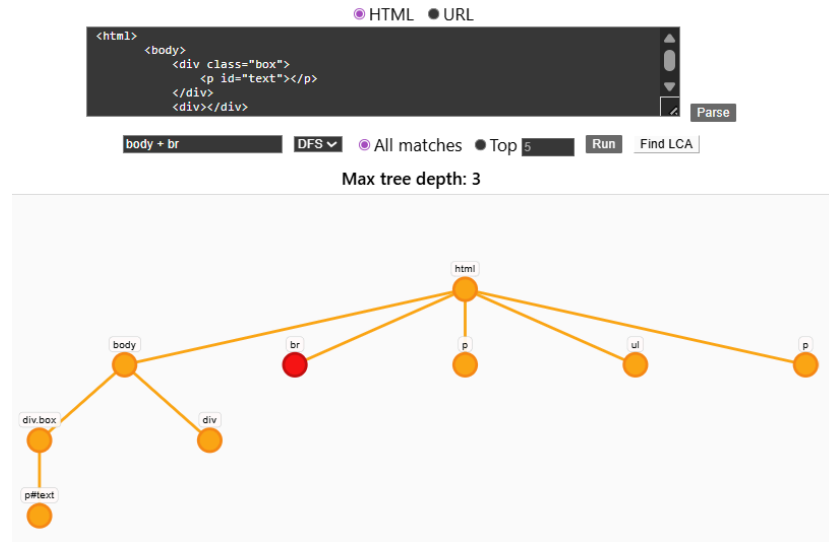
Gambar 4.2.e_5. Pengujian Child Combinator DFS

vi. Descendant Combinator



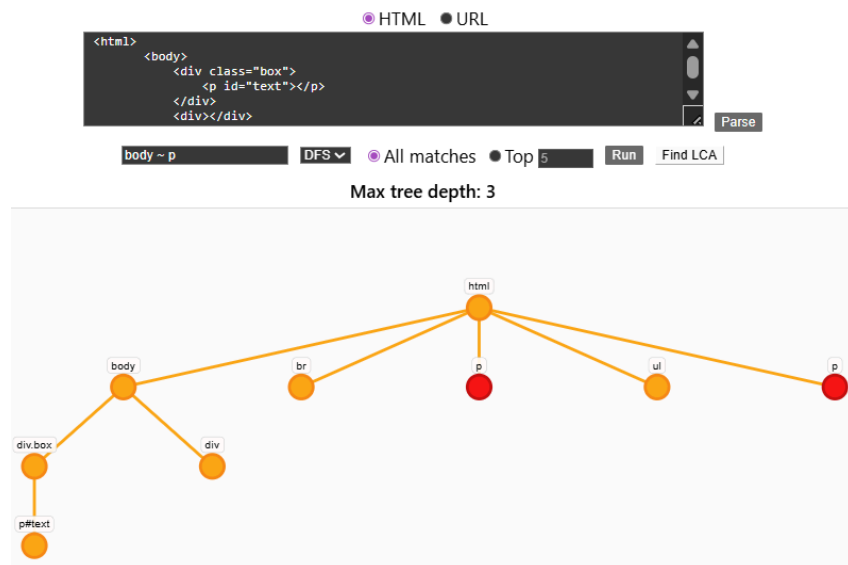
Gambar 4.2.e_6. Pengujian Descendant Combinator DFS

vii. Adjacent Sibling Combinator



Gambar 4.2.e_7. Pengujian Adjacent Sibling Combinator DFS

viii. General Sibling Combinator



Gambar 4.2.e_8. Pengujian General Sibling Combinator DFS

4.3. Analisis Algoritma

Breadth First Search (BFS) dan Depth First Search (DFS) merupakan dua algoritma traversal graf yang digunakan untuk menelusuri struktur data seperti DOM Tree dalam proses pencarian elemen berdasarkan CSS selector. Algoritma BFS bekerja

dengan menelusuri setiap node pada tingkatan yang sama dan melanjutkan pencariannya pada tingkatan berikutnya. Secara teori, algoritma BFS merupakan algoritma exhaustive sehingga memiliki kecenderungan untuk menggunakan lebih banyak memori dibandingkan DFS. Sifat BFS yang menelusuri seluruh node berkontribusi pada penggunaan memori ini. Hal ini bisa terlihat dari penggunaan queue pada source code. Algoritma BFS juga memiliki keunggulan dalam menemukan tag yang muncul pada tingkatan atas terlebih dahulu di dalam tree akibat sifat exhaustive-nya.

Di sisi lain, Depth First Search (DFS) melakukan traversal secara mendalam dengan menelusuri satu cabang hingga mencapai node terdalam sebelum berpindah ke cabang lainnya. DFS memiliki logika backtrack untuk ke node lainnya sebelum memasuki daun yang sudah mati. Akibatnya, DFS menggunakan memori yang relatif sedikit. Namun, DFS akan mencari node terdalam untuk menentukan suatu pencarian tag, sehingga ketika dibandingkan pencarian top N dengan DFS dan BFS akan juga relatif berbeda.

Pada tugas ini, preferensi penggunaan BFS dan DFS bergantung pada kasus pencarian. BFS cocok digunakan ketika ingin mencari elemen yang paling dekat dengan root, sedangkan DFS lebih efektif untuk eksplorasi mendalam atau ketika seluruh kemungkinan elemen perlu diperiksa. Algoritma BFS dan DFS hanya berpengaruh pada Top N kemunculan.

Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Pada proyek Tugas Besar 2 IF2211 Strategi Algoritma, tim berhasil mengimplementasikan sebuah CSS selector untuk DOM dengan memanfaatkan struktur pohon sebagai representasi data. CSS selector ini telah diimplementasi oleh tim memakai algoritma Breadth First Search (BFS) dan Depth First Search (DFS) serta Depth Limited Search (DLS) dalam pencarian tag yang sesuai. Tujuan utama dalam tugas besar ini adalah mencari HTML tag berdasarkan CSS selector pada struktur DOM yang bersifat hierarkis, sehingga pendekatan traversal graf menjadi solusi yang tepat untuk mencari node satu per satu.

Selector ini juga memerlukan beberapa tahapan, yakni pemrosesan input dokumen, pemrosesan query, pencarian elemen, dan penampilan output. Tahapan pertama, pemrosesan input dokumen, bisa dilakukan dengan dua metode yaitu memasukkan file .html atau dengan memasukan link URL yang akan difetch. Input kemudian diproses dengan HTML parser. Begitu juga dengan query untuk selector yang akan diparsing untuk mencocokkan logika node. Elemen kemudian dicari sembari query diparsing dan ditampilkan melalui Web App yang sudah diimplementasikan. Dengan memanfaatkan struktur DOM Tree tersebut, tim berhasil memahami algoritma BFS, DFS, dan DLS yang dapat digunakan untuk menelusuri dan mencocokkan elemen terhadap query CSS selector.

5.2. Saran

Untuk pengembangan lebih lanjut, source code dapat di-*improve* melalui saran-saran berikut ini. Source code dapat dikembangkan lagi lebih jauh dengan implementasi selector yang lebih kompleks dengan pencarian atribut seperti selektor asli dari referensi luar. Selain itu, performa sistem juga dapat dioptimalkan, baik dari segi efisiensi traversal maupun pengelolaan memori, terutama saat menangani DOM

dengan ukuran besar. Di sisi lain, masih terdapat ruang untuk pengembangan UI/UX dengan desain yang lebih interaktif lagi.

Lampiran

Github Repository: https://github.com/BillyOWasTaken/Tubes2_IdkWhatYet

Pernyataan Karya Orisinalitas

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Jennifer Khang



Billy Ontoseno Irawan



Ahmad Rinofarus Mochtar

Tabel Penyelesaian

No	Poin	Ya	Tidak
1	Aplikasi berhasil dikompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Tabel Pembagian Tugas

Nama	Tugas
Jenka	Tree, Web Scraper, HTML Parser, CSS Selector, DFS, BFS, Laporan
Billy	Frontend, Integration, Docker, Laporan (Arsitektur)
Rino	LCA, Laporan, Testing

Daftar Pustaka

1. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf)
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press